

bomblab 报告

姓名：赵子涵

学号：2024201579

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	0	0	0	0	0	0	0

scoreboard 截图：

2024201579	0	0	0	0	0	0	0	0	0	0	0	0	0	0	7
------------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

解题报告

phase_1

Just let it all out of you. Don't be swayed!

解题思路：

1. 查看汇编代码

```
0x55555555435 <phase_1>      sub    $0x8,%rsp
0x55555555439 <phase_1+4>    lea     0x1d40(%rip),%rsi      # 0x555555557180
0x55555555440 <phase_1+11>  call   0x55555555d13 <strings_not_equal>
```

可得，要对输入进行字符串比对，又 %rsi 为传入函数 <strings_not_equal> 的参数。

2. 查看其内容

```
(gdb) x/s 0x555555557180
0x555555557180: "Just let it all out of you. Don't be swayed"
```

如此便得到此题答案。

phase_2

556596 356625 531767 388658

解题思路：

1. 代码开始时调用 `sscanf` 函数，%rsi 中存储的即为输入第一个参数——格式参数。

```
0x5555555547d <phase_2+40>    lea     0x214d(%rip),%rsi      # 0x5555555575d1 |
0x55555555484 <phase_2+47>  call   0x55555555150 <__isoc99_sscanf@plt>
```

查看 %rsi 内容，同时下文有检查程序，得知输入为4个 `int` 整数。

```
(gdb) x/s $rsi
0x5555555575d1: "%d %d %d %d"
```

```
0x55555555489 <phase_2+52>    cmp    $0x4,%eax
0x5555555548c <phase_2+55>    jne    0x555555554a2 <phase_2+77>
```

2. 阅读后续代码，可以看出读入了 `matA`、`matB`

```
(gdb) x/6dw 0x55555555a140
0x55555555a140 <matA.3>:      165    399    402    117
0x55555555a150 <matA.3+16>:  586    162
(gdb) x/6dw 0x55555555a120
0x55555555a120 <matB.2>:      85    514    713    515
0x55555555a130 <matB.2+16>:  642    165
```

3. 代码中主要内容为二重循环，其中

```
matA in %rdi, matB in %rsi
```

```
while(r8d < 2){
    while(rax < 3){
        edx = (rdi + 4 * rax) * (rsi + 8 * rax);
        ecx += edx;
        rax++;
    }
    r8d++;
}
```

可分析得出程序是在进行两个矩阵的乘法， 2×3 与 3×2 相乘，结果为 2×2

手动计算结果，可得此题答案.

phase_3

```
2 D 185
```

解题思路:

1. 开始思路相似，得到输入格式

```
(gdb) x/s 0x5555555571e5
0x5555555571e5: "%d %c %d"
```

2. 输入的 `%c` 进行了与 `mask = 0x20` 异或的操作.

```
0x5555555557e <phase_3+58>    xor    %al,0xf(%rsp)
```

3. 由多段重复代码，在其之前还有 `jmp *%rax`，可初步判断是 `switch` 跳转结构。

而此处的 `*%rax`，正是由初始输入的第一个 `%d` (`in 0x10(%rsp)`) 决定的.

```

0x555555558d <phase_3+73>    mov     0x10(%rsp),%eax
0x5555555591 <phase_3+77>    lea     0x1c88(%rip),%rdx      # 0x555555557220
0x5555555598 <phase_3+84>    movslq (%rdx,%rax,4),%rax      0x5555555559c
<phase_3+88>    add     %rdx,%rax      0x5555555559f <phase_3+91>
jmp     *%rax

```

每个 case 中，有比较此 case 中数值与输入的第二个 %d 的程序。

```

0x55555555f1 <phase_3+173>    cmp     $0xb9,0x14(%rsp)

```

在所有 case 的跳转终点，有一个比较判断，即比较 case 中字符与输入的字符。

```

0x555555556a3 <phase_3+351>    cmp     %a1,0xf(%rsp)

```

4. 最后的标准答案，只要 case 编号、掩码修饰前的字符、case 内数值对应即可，即答案不唯一，共有8组。

phase_4

31 BA

解题思路：

1. 开始思路相似，得到输入格式

```

(gdb) x/s 0x5555555571ee
0x5555555571ee: "%d %2s"

```

2. 对 func4_1 进行分析可得如下程序 (x in %edi)

```

int func4_1(int x) {
    if (x <= 0) return 0;
    if (x == 1) return 1;
    x--;
    func4_1(x);
    x = 2 * x - 1;
}

```

可得本题输入 "%d" 应为 func4_1(5) = 31。

3. 再观察本题另一个判断程序

```

0x555555557ff <phase_4+118>    lea     0x10(%rsp),%rdi
0x55555555804 <phase_4+123>    mov     %rbx,%rsi
0x55555555807 <phase_4+126>    call    0x55555555d13 <strings_not_equal>
0x5555555580c <phase_4+131>    test    %eax,%eax
0x5555555580e <phase_4+133>    jne     0x5555555583b <phase_4+178>

```

在 <strings_not_equal> 处打断点，检查所比对的字符串

```
(gdb) x/s $rsi
0x7fffffffda44: "BA"
```

得到本题另一个输出 "%2s" 即为 "BA".

phase_5

acfhij

解题思路:

1. 要求输入长度为6的字符串.
2. 分析汇编得到大致程序代码

```
int *p = input;
for (int i = 0; i < 6; i++){
    int t = *p;
    t %= 16;
    int ans = *(arr + t); // arr为 %rsi 中的 array
}
```

3. 由后文结果对比程序

```
0x5555555587a <phase_5+51>    cmp    $0x2d,%ecx
0x5555555587d <phase_5+54>    jne    0x55555555888 <phase_5+65>
```

应满足 `ans = 45`. 查看 `array` 内容

```
(gdb) x/16dw 0x55555557240
0x55555557240 <array.0>:      2      10      6      1
0x55555557250 <array.0+16>:    12      16      9      3
0x55555557260 <array.0+32>:    4       7     14      5
0x55555557270 <array.0+48>:    11      8     15     13
```

即其中6个数字之和应为45, 选出10、1、9、4、7、14. 他们分别是 `arr[1]`、`arr[3]`、`arr[6]`、`arr[8]`、`arr[9]`、`arr[10]`, 数组的 `index` 都是 `mod 16` 的结果.

以 `ASCII = 96` 为基准选取6个字符: `acfhij`, 即为答案.

phase_6

```
3 5 4 1 2 6
# 若要跳转到secret_phase
3 5 4 1 2 6 binary
```

解题思路:

1. 输入6个数字, 可以看出数字不能超过6.
2. 进行重复检查 (`%r13`、`%r13`、`%r14` 都是输入六个数字的备份)

```

0x555555558da <phase_6+75>    add    $0x1,%rbx
0x555555558de <phase_6+79>    cmp    $0x5,%ebx
0x555555558e1 <phase_6+82>    jg     0x5555555598e <phase_6+255>
0x555555558e7 <phase_6+88>    mov    0x0(%r13,%rbx,4),%eax
0x555555558ec <phase_6+93>    cmp    %eax,0x0(%rbp)

```

3. 用7减去输入的数字

```

0x555555558f8 <phase_6+105>   mov    0x8(%rsp),%rdx
0x555555558fd <phase_6+110>   add    $0x18,%rdx
0x55555555901 <phase_6+114>   mov    $0x7,%ecx
0x55555555906 <phase_6+119>   mov    %ecx,%eax
0x55555555908 <phase_6+121>   sub    (%r12),%eax
0x5555555590c <phase_6+125>   mov    %eax,(%r12)
0x55555555910 <phase_6+129>   add    $0x4,%r12
0x55555555914 <phase_6+133>   cmp    %r12,%rdx
0x55555555917 <phase_6+136>   jne    0x55555555906 <phase_6+119>

```

4. 按照所得数列匹配节点，然后把所有节点连接起来。

```

; 匹配节点过程
0x55555555933 <phase_6+164>   mov    0x8(%rdx),%rdx
0x55555555937 <phase_6+168>   add    $0x1,%eax
0x5555555593a <phase_6+171>   cmp    %ecx,%eax
0x5555555593c <phase_6+173>   jne    0x55555555933 <phase_6+164>
0x5555555593e <phase_6+175>   mov    %rdx,0x30(%rsp,%rsi,8)
0x55555555943 <phase_6+180>   add    $0x1,%rsi
0x55555555947 <phase_6+184>   cmp    $0x6,%rsi
0x5555555594b <phase_6+188>   jne    0x5555555591e <phase_6+143>

```

5. 最后比较节点的 `value`，必须是降序排列。

6. 逆向推导本题答案：

- 我用 1 2 3 4 5 6 进行实验推导。
- 先查看各节点对应 `value`，发现 `node6` 找不到，而在把所有节点连接的最后一步，连接的就是 `node6`。
- 按照值的顺序排列 1~6，在用7减去，得到原本输入，即为答案。

secret_phase

3301252

解题思路：

1. 经观察汇编代码可得，`secret_phase` 位于 `<phase_defused>` 中

```

0x55555556228 <phase_defused+117>    movslq %esi,%rsi
0x5555555622b <phase_defused+120>    lea    0x4766(%rip),%rax            #
0x5555555a998 <input_strings+600>
0x55555556232 <phase_defused+127>    lea    (%rsi,%rax,1),%rdi
0x55555556236 <phase_defused+131>    lea    0x13e4(%rip),%rsi            #
0x55555557621
0x5555555623d <phase_defused+138>    call   0x55555555d13 <strings_not_equal>

```

可见需要进行对 phase_6 的输入进行字符串比对，比对内容即为 `secret word = binary`。把 `secret word` 放在 phase_6 之后即可找到 `secret_phase`。

2. 主体存在于 `func7` 之中，分析可以得到以下内容：

```

// 4组方向数组
dx1[8] = {-2, -1, 1, 2, 2, 1, -1, -2};
dy1[8] = { 1,  2, 2, 1,-1,-2, -2, -1};
dx2[8] = {-1,  0, 0, 1, 1, 0,  0, -1};
dy2[8] = { 0,  1, 1, 0, 0,-1, -1,  0};

// 通过row0，可以找出一个8*8"地图"。
row[0] = {0, 0, 1, 0, 0, 1, 0, 0};
row[1] = {0, 0, 0, 1, 0, 0, 0, 1};
row[2] = {1, 0, 1, 0, 0, 1, 0, 0};
row[3] = {1, 0, 0, 0, 0, 0, 0, 0};
row[4] = {0, 1, 0, 0, 1, 0, 1, 0};
row[5] = {1, 0, 0, 1, 1, 0, 0, 0};
row[6] = {0, 0, 0, 0, 0, 1, 0, 1};
row[7] = {0, 1, 0, 0, 0, 0, 0, 0};

```

3. 由最开始的 `%esi` 和 `%edx`，可得起点为 (0, 0)；程序最外层有判断条件，可得终点为 (7, 4)。

```

0x55555555b16 <func7+286>    cmp     $0x4,%esi
0x55555555b19 <func7+289>    jne     0x55555555b86 <func7+398>
0x55555555b1b <func7+291>    cmp     $0x7,%edx
0x55555555b1e <func7+294>    jne     0x55555555b86 <func7+398>

```

4. 程序中有两部分移动与判断函数

```

// "马走日"
r8d = a + dx1[dir]
r11d = b + dy1[dir]
// 边界检查
eax += dx2[dir]
edx += dy2[dir]
// 两次可视为
na2 = a + dx1[dir] + dx2[dir];
nb2 = b + dy1[dir] + dy2[dir];

```

5. 程序用递归实现 DFS 进行路径搜索，针对地图和移动方式设计算法，找出符合条件的路径，即得出答案 "3301252"。

反馈/收获/感悟/总结

- **反馈：**这个Lab花费时间很长，主要的难点是在于，每个程序，都有大量的寄存器和跳转调用语句，经常容易搞不清楚谁是谁，容易跟丢。就算勉强转化为能理解的代码，也很难直接看出它表达出的思路 and 核心操作。
- **总结：**需要学会“忽略”大部分汇编，聚焦于核心功能性语句，尤其是计算和调用之类。
- **收获：**因为我的报告写的较迟，在中途，程序曾因实验过期出现强行停止 ("Lab expired")。我学习如何通过 gdb 去把汇编中的某些语句NOP掉，e.g. 我把其中 `"call <send_msg>"` NOP掉，阻止程序向 `score board` 发送消息，从而避免了程序中断。